Comparing self-extracted to third-party audio features for music genre classification

Bradley Erickson
Winona State University
BErickson15@winona.edu

ABSTRACT

There is no definitive way to determine what genre music falls into. Conducting an analysis into the different audio features could prove useful in creating an automated process for determining genre. Using self-extracted audio features and third-party audio features, we create two artificial neural networks that will classify the song genre. Comparing our models with test data, we find self-extracted features do a better job at predicting genre.

KEYWORDS

genre, classification, neural network

1 BACKGROUND

Song genres are primarily relative to the listener and there is no clear-cut way to classify which genre a song belongs to. With the power of machine learning, researchers have taken a crack at automating this process using artificial neural networks. Conducting this and other audio analysis can prove useful to music companies that wish to understand what customers enjoy the most. Other researchers have created convolutional neural networks to classify music by training on chunks of sound instead of various features [1–3]. For comparing data, two mirror artificial neural networks (ANN) will be created. One trained on the features extracted directly from the audio and the other trained using third-party song metrics. The two models will be compared to see which method has the better prediction accuracy.

2 DATA

We will focus on 10 musical genres, or categories, for classification:

(1) Blues
(2) Classical
(3) Country
(4) Disco
(5) Hip-hop
(6) Jazz
(7) Metal
(8) Pop
(9) Reggae
(10) Rock

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.
Proceedings of 20th Winona Computer Science Undergraduate Research Seminar, May 1, 2020, Winona, MN

2.1 Self-extracted

Our first model will train using the GTZAN dataset which was created for extracting features and classifying music into genres [4]. This dataset contains 1,000 30 second song clips spanning across the 10 genres. We will use visual representation of frequencies, or spectrograms, to extract various features from the audio file. The following metrics will be extracted and used as input for our model:

(1) **Zero crossing rate**
Description: Rate of signal sign changes
Usage: Good for detecting percussive sounds
(2) **Chroma Shift**
Description: Map hertz to binned pitch class
Usage: What notes are being played
(3) **Spectral centroid**
Description: Brightness of tone
Usage: Distinguish between sounds
(4) **Spectral Bandwidth**
Description: Range of hertz
Usage: Specific frequency at interval
(5) **Spectral Rolloff**
Description: Central frequency
Usage: Approximates maximum or minimum frequency
(6) **Tempo**
Description: Estimated beats per minute
Usage: Determine speed of song
(7) **Root mean square**
Description: Perceived loudness of clip
Usage: Overall volume of track
(8) **Mel-frequency cepstral coefficients**
Description: Magnitude of shortened clip
Usage: Good for detecting level of voices

2.2 Third-party

For our second model, we will use data scraped from the Spotify API [5]. With this API, we will gather song metrics for inputs to our model and genres to for the output. Spotify does not include information about how they determine or detailed descriptions of the metrics they provide. The following metrics with brief descriptions will be used:

(1) **Key**
Description: Estimated key using standard pitch class
(2) **Mode**
Description: Major or minor
(3) **Time signature**
Description: Estimated time signature
(4) **Acousticness**
Description: Confidence of track being acoustic, non-electric

(5) **Danceability**
Description: How suitable a track is for dancing
(6) **Energy**
Description: Perceptual measure of intensity and activity
(7) **Instrumentalness**
Description: Confidence a song has no vocals
(8) **Liveness**
Description: Presence of audience
(9) **Loudness**
Description: Overall loudness in decibels
(10) **Valence**
Description: Musical positiveness
(11) **Tempo**
Description: Estimated tempo in beats per minute

3 METHODS

3.1 Data manipulation

The self-extracted data will be pulled from a spectrogram. An example spectrogram is shown in Figure 1. We will create a spectrogram for each song clip in the GTZAN dataset. Using the Librosa package in Python, we will extract our desired features and store them in a csv file. To obtain the third-party data, we query the top 100

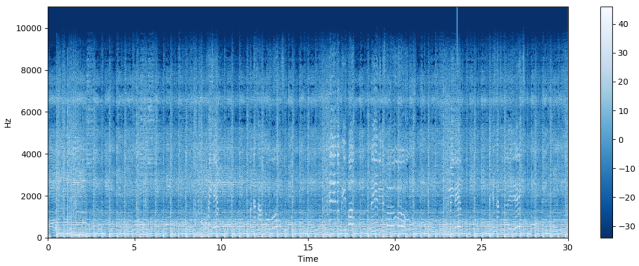

Figure 1: Example spectrogram. Color corresponds to intensity of sound.

songs for each of our genres using the Spotify API. For each song returned, we query our desired features. We store the features and genre label in a csv file. With both of our datasets, we transform the each feature to a normal distribution using Yeo-Johnson power transformations. Additionally, we will scale the distributions to use a [0, 1] range. Figure 2 shows a feature before and after the data manipulations.

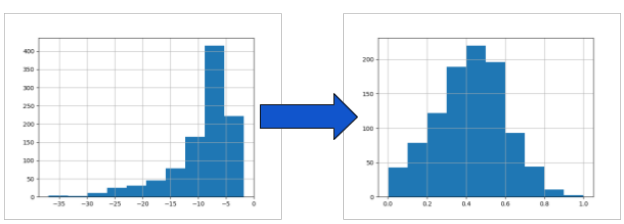

Figure 2: Example feature before and after transformation.

3.2 Model creation

With how complex music is, we want our classifier to have a deeper understanding of the inputs. This can be accomplished with an ANN, artificail neural network. Since, ANNs are a field of complex neurons and each input touches each neuron, this will allow our models to learn about relationships between our inputs. To compare the datasets, we will create two ANNs that are mirrored in structure. Our final model is shown in Figure 3 We start with a 64-node hidden layer with a ReLU output to provide a large initial layer on our model. We add a batch-normalization layer to keep the mean outputs of the first layer close to 0 with a standard deviation close to 1. This will help to not overfit the training data. We add a 32-node and a 16-node layer, both using ReLU as output. Lastly, we have a 10-node layer with a softmax output. One node for each genre.

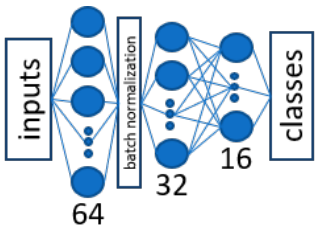

Figure 3: Neural network model structure.

3.3 Training

We split the data into training and test sets using an 80%-20% split, stratifying on genre. With each training dataset, we will conduct Monte-Carlo cross-validation to train 20 models. We will train using 20 epochs and a 64-unit batch size. These parameters yielded the best performance without overfitting. Finally, we will average the validation accuracy from each set of 20 models for comparison.

4 RESULTS

Figure 4 shows the self-extracted test data's actual versus predicted results for their model. This model did an outstanding job predicting data in the classical, metal, and pop genres. On the other hand, this model did a poor job predicting data in the rock genre, classifying 40% of the data-points as disco.

Similarly, Figure 5 shows the third-party test data's actual versus predicted results. This model only did well classifying data-points in the classical genre. This model did a poor job classifying data in the blues, pop, and rock genres. We notice that data belonging to pop and hip-hop are often misclassified as one another.

Table 1 shows the average validation accuracy and the test accuracy for each of our models. The self-extracted model correctly classified the validation data, on average, 6.32% better than the third-party model. For the test data, the self-extracted model correctly classified by 13% better than the third-party model. These results show the self-extracted data is better for predicting genre in a neural network than the third-party data.

5 FUTURE WORK

There are a few ways we can improve this work. First, we could use different classification methods, such as k-nearest neighbors or

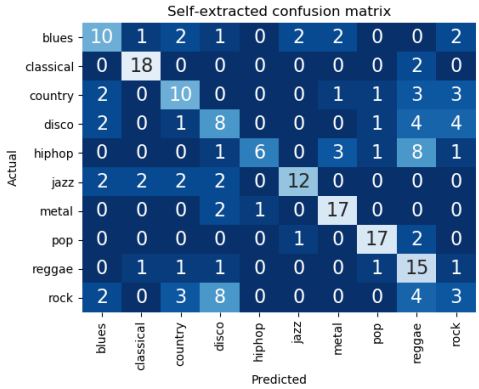

Figure 4: Self-extracted model confusion matrix.

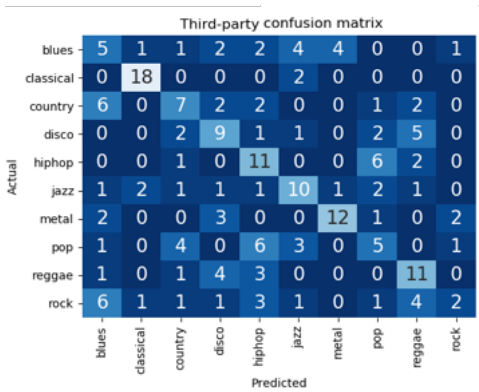

Figure 5: Third-party model confusion matrix.

Table 1: Final accuracy percentages

	Avg. validation accuracy	Test accuracy
Self-extracted	52.97%	58%
Third-party	46.65%	45%

a tree-based method. Additionally, we could expand the datasets to cover other genres and include more songs in our current genres. Lastly, the best option for comparison would be to use the same songs in each dataset.

6 REFERENCES

REFERENCES

[1] Ahrendt, Peter, et al. *Decision Time Horizon for Music Genre Classification Using Short Time Features.* 6 Sept. 2004.

[2] Jeong, Il-Young, and Kyogu Lee. *Learning Temporal Features Using a Deep Neural Network and its Application to Music Genre Classification.* Ismir. 2016.

[3] Li, Tom LH, Antoni B. Chan, and Andy HW Chun. *Automatic musical pattern feature extraction using convolutional neural network.* Genre 10 (2010): 1x1.

[4] Tzanetakis, George, and Perry Cook. *Musical genre classification of audio signals.* IEEE Transactions on speech and audio processing 10.5 (2002): 293-302.

[5] Spotify for Developers API. https://developer.spotify.com/documentation/web-api/